

7 利用 Socket API 实现许可认证软件

7.1 实验目的

通过完成实验，掌握应用层文件传输的原理；了解传输过程中传输层协议选用、应用层协议设计和协议开发等概念。

7.2 需求说明

某远程桌面连接软件 A 需要有一个许可证程序来保证合法运行。这样的许可证 (License) 是带有限制的。用户提供支付费用，获得一个若干人 (如：10 人或 50 人) 的许可证。规则如下：

- 1、某组织管理员在购买许可证时，输入用户名、口令和许可证类型，许可证程序返回一个由 10 个数字组成的序列号。
- 2、该组织的用户第一次使用软件 A 时，输入序列号。
- 3、该组织用户运行软件时，向许可证服务器发送验证。
- 4、许可证服务器查询得到该序列号的使用人数，如果未到达上限，则返回授权指令；否则，返回拒绝指令。
- 5、软件 A 得到授权指令，允许用户使用软件。否则，提示用户稍后再试，退出程序。

当软件 A 或非正常退出 (崩溃被其它程序中止) 时，许可证服务器应在扣除使用人数时剔除它。可以使软件 A 定期 (如：30 分钟) 向服务器报告其状态，超过一定时间没有收到报告时，则认定崩溃。

当许可证服务器崩溃时，软件 A 应能在重新启动时恢复。许可证服务器重启后，如果新的软件 A 前来连接，服务器不可以因接受其连接而拒绝已认证用户的连接。

7.3 输入输出接口

程序应提供 Windows 控制台可执行程序接口，以便于测试。

7.4 技术建议

- 1、本实验自选开发语言，但推荐 C 和 C++；
- 2、服务器的网页接口部分推荐 PHP，用于表达数据。
- 3、了解基本的动态文档网页开发。

7.5 附录一：编写许可证服务器

第 13 章 基于数据报(Datagram)的编程：编写许可证服务器^①



概念与技巧

- 基于数据报的编程,数据报 socket
- TCP 与 UDP
- 许可证服务器
- 软件时间戳(Software ticket)
- 设计健壮系统
- 设计分布式系统
- Unix 域的 socket

相关的系统调用与函数

- socket
- sendto,recvfrom

13.1 软件控制

程序的运行需要内存、CPU 和一些系统资源。操作系统关心这类事情,但是一些程序还需要关心另一件事情:就是程序拥有者的允许。

从合法性的角度讲,需要有一个许可证(license)来保证程序的合法运行,但是有些许可证却是带有限制的。例如,有的许可证是限制同时运行程序的用户数。一个 10 人的许可证可能需要一定的花费,而 50 人的许可证可能需要更多的花费。有些厂商租赁软件许可证,当租赁期到了,程序就不能继续运行。当然除了合法性方面之外,软件也还受到其他一些因素的限制。学校里的计算机房就可能限制每天游戏程序运行的次数。

有些软件拥有者使用诚信机制来限制程序的使用,他们把许可证条款打印在屏幕上或纸上并要求用户遵守协约。其他的则使用特定的技术来实施许可证条款。

^① 本章的内容基于 Lawrence deLuca 在哈佛职业教育学院担任助教期间编写的讲稿,该讲稿取材于他参与开发的一个产品。

一种实施许可证技术是编写程序来执行许可证控制。通常的做法是设计从一个许可证服务器获得许可的应用程序。该服务器是一个进程,它授权应用程序的运行。

许可证服务器明确许可证条款并且强制执行该条款,如图 13.1 所示。

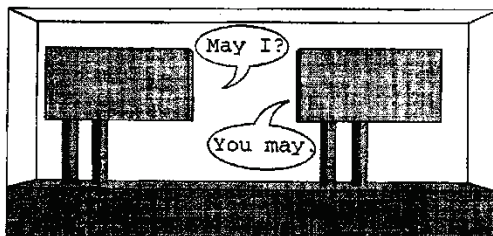


图 13.1 许可证服务器给予许可

请求许可和给予许可需要客户和许可证服务器之间的通信。

许可证服务器是如何工作的?本章将学习一个具体的客户/服务器许可证控制模型。通过该模型的学习,可以接触到另外一种类型的 socket——数据报 socket,另外的一个地址域——Unix 域,一个维持系统状态的网络协议,以及其他一些有关设计安全健壮的客户/服务器系统的技术。

13.2 许可证控制简史

软件控制技术的使用已经有多年的发展历史了。

在单机版的个人计算机时代,对软件的限制是靠特定的磁盘或由隐匿在特定磁道上的密码来实现的。磁盘上的密码很难被复制,并且只有磁盘在驱动器中的时候程序才能运行。如果磁盘丢失了或被损坏了,该程序将不能再运行。人们很快就破解并找到了如何来复制这种特殊磁盘的方法,所以软件厂商发明了硬件密钥。硬件密钥是一个适配器,它可以插在并口、串口或 USB 口上;被许可的程序只有在发现该适配器的时候才能运行。如果硬件密钥丢失,程序将不能运行。

然而网络计算机和多用户系统却引起了新的问题。如果 10 个用户同时想运行计算机或网络上的同一个程序,那么是不是每个用户都需要在服务器的端口上插一个硬件密钥呢?软件厂商们需要一种可靠的并且不需要给合法用户增加额外负担的方法来实施许可证条款。

网络和多用户系统提供了一种新的解决方法:许可证服务器。程序不是从磁盘或密钥取得许可,而是从服务器进程取得。许可证服务器被一台计算机上多个用户共享,服务器进程能够控制程序的使用人数、使用时间、使用地点甚至程序的使用方式。伴随更多的计算机加入因特网,服务器控制对软件和数据访问的需求也更加迫切。

本章中的许可证服务器将实施 n 用户的限制。也就是说,服务器只允许特定数量的程序实例同时运行。

13.3 一个非计算机系统实例: 轿车管理系统

一个公司购买了许可证,该许可证限制了同时使用程序的用户数。公司可能拥有比许可证所允许的用户要多的雇员,但是并非所有的雇员要同时使用程序。怎样的一个设计才能满足上面的需求呢?

现实世界中有很多系统,通常由一大群人共享其中的资源,而这些资源是有限的。这里将分析一个模型:雇员共享公司轿车的问题。一个公司拥有特定数量的轿车,同时还拥有更多数量的雇员。如何控制对轿车的使用呢?

13.3.1 轿车钥匙管理描述

控制轿车的使用是通过控制对轿车钥匙的访问。当想用轿车的时候,必须先得到一把钥匙。如果在钥匙盒中没有钥匙了,将不能使用轿车。如果有可用钥匙,可以先拿一把钥匙,签名,然后使用轿车。使用完毕后,把钥匙放回钥匙盒,在使用轿车名单中划去自己的名字。过程描述如图13.2所示。

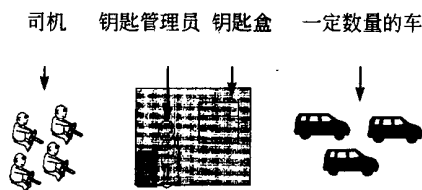


图 13.2 控制对轿车的的使用

签名列表有何作用呢?该系统的目的是通过控制对轿车的使用,使得可用的钥匙一直很充足。人并非完美的,有时司机会忘记归还钥匙。钥匙管理员可以根据签名列表找到该司机,确定他是否仍在用车。

轿车使用管理系统是控制软件使用的一个现实模型。在将该系统转换成软件系统之前,需要更为详细地描述该系统。

钥匙管理系统的构成如下:

- (1) 钥匙管理中心的地点——到哪儿可以获得钥匙
- (2) 钥匙管理员——执行策略的人
- (3) 钥匙——需要得到的东西
- (4) 签名列表——保存钥匙和找回钥匙的记录

13.3.2 用客户/服务器方式管理轿车

在给出轿车钥匙管理系统的构成后,下面将用客户/服务器语言来描述该系统。

(1) 服务器和客户

谁是服务器和谁是客户?钥匙管理员拥有司机需要的钥匙。用网络术语来描述,钥匙

管理员是服务器,司机是客户。

(2) 协议

所用的协议是什么? 交互的事务是什么? 轿车钥匙管理协议包括两个主要的事务。

• 获得钥匙

客户: 你好,我需要一把钥匙。

服务器: 这里只有 5 号钥匙,没有别的钥匙了。

• 归还钥匙

客户: 我已经用好 5 号钥匙。

服务器: 谢谢。

(3) 通信系统

客户和服务器如何通信? 在该系统中,人们通过对话来传递简单信息。

(4) 数据结构

司机和钥匙管理员需要什么样的数据结构呢? 钥匙管理员保留一个用户签名列表,一把钥匙对应列表的一项。当一个用户取走一把钥匙,钥匙管理员在该项记下用户的名字,当用户归还钥匙时,管理员把司机的名字从列表中擦去。下表解释了用户签名列表:

签名列表	
钥匙 #	司 机
1	adam@sales
2	
3	carol@support
4	

如果在签名列表中没有司机与钥匙号对应,表示该钥匙是可用的。反之,表示不可用。

13.4 许可证管理

本节将钥匙管理系统的思想运用到许可证管理系统中。

13.4.1 许可证服务系统: 它做些什么

图 13.3 描述了人们试图运行许可证程序的过程。工作如下:

- (1) 用户 U 运行被许可的程序 P;
- (2) 程序 P 向服务器 S 请求运行许可;
- (3) 服务器检查当前运行程序 P 的用户数;
- (4) 如果上限未达到,S 给予许可,程序 P 运行;
- (5) 如果达到上限,S 拒绝许可,程序 P 告诉 U 稍后再试。

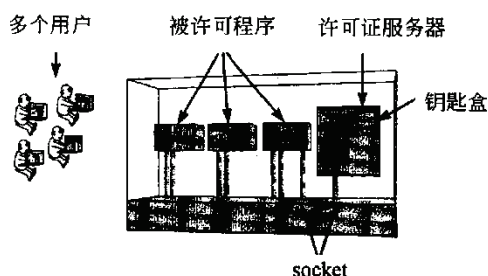


图 13.3 控制软件的使用

许可证服务系统与轿车钥匙服务系统稍有不同。在轿车系统中,司机向钥匙管理员请求许可;而在这里,程序向服务器请求许可。这就像司机请求轿车,而轿车向钥匙管理员请求钥匙^①。

程序的创建者要编写所有的程序:应用程序和服务端。这两个程序作为一个系统。服务器给予应用程序运行许可和实施许可证条款。如果许可证服务器不在运行,应用程序将得不到许可,不能运行。

这里以轿车钥匙服务系统作为模型进行了讨论。那么如何将这种思想运用到软件系统中呢?被许可程序如何从服务器得到许可?服务器如何给予许可?软件系统和轿车钥匙系统的等价之处在哪里呢?

13.4.2 许可证服务系统:如何工作

(1) 票据模型

钥匙管理员负责分发钥匙,许可证服务器发布什么呢?以电影院和棒球场为例,付钱获得进场许可,然后得到一张入场票据。与此类似,这里的许可证服务器发布的是数字票据。这种票据又是什么样子的呢?客户和服务端交换字符串,所以票据是字符串,其格式如下:

pid.ticketnumber 例如: 6589.3

每张票据包括持有该票据进程的 PID 以及票据编号。在票据中包含 PID 的原因其实就和在飞机票上印上名字的道理是一样的。票据中的 PID 标识票据的使用者,在票据丢失时,可以帮助找回票据。进程可能丢失票据吗?

(2) 服务器和客户

谁是客户端和谁是服务器?许可证服务器持有程序需要的资源:票据。用网络术语,许可证服务器是服务器,而应用程序是客户。

(3) 协议

协议是什么?交互的事务是什么?这里的票据管理协议包括下面的两个主要事务:

^① 这个概念并不难理解。随着设备越来越先进,连接越来越方便,很可能在不久的将来就可以通过你的收音机询问服务器是否有播放你最喜欢歌曲的许可。

- 获取钥匙

客户: HELO mypid

服务器: TICK ticketid or FAIL no tickets

- 归还钥匙

客户: GBYE ticketid

服务器: THNX message

这里定义了基于文本的协议,协议中使用了 4 个字符的简单命令,这与前面的 Web 服务器所用的命令类似。

(4) 通信系统

上述简短的文本信息如何在客户和服务器之间传送? 在本文后面将讨论该问题。

(5) 数据结构

客户和服务器之间所用的数据结构是什么? 这里使用整型数组作为签名列表。数组的每个入口对应一张票据。当一个客户取走了一张票,管理员将客户对应的 PID 写到该入口。如下表:

签名列表	
tick #	process
1	1234
2	0
3	6589
4	0

如果数组中的某个元素值是 0,表明该票据可用。否则,表明该票据正在被使用。

13.4.3 一个通信系统的例子

客户如何请求票据? 服务器如何发布票据? 这涉及到进程间的通信形式。客户和服务 器之间通过短消息通信。服务器必须接收,处理和应答来自多个客户的请求。目前,哪些技术是可以使用的呢? 本文前面学习的信号和管道机制可以考虑,但是信号太短了,而管道只连接相关联的进程。所以,使用 socket 是最明显的答案。而对于 socket,也有两种不同的选择。一种是基于流的 socket,它是用来连接不相关的进程的。另一种 socket 被称为数据报 socket,或称为 UDP,对于现在的项目,这种 socket 是更好的选择。

13.5 数据报 socket

流 socket 传送数据就跟电话网中传送声音一样,客户先建立连接,然后使用该连接进行单向、双向或类似管道的字节流传送。

数据报通信则与从一个邮箱到另一个邮箱发送包裹类似。客户不必建立连接,只要向特定的地址发送消息,而服务器进程在该地址接收消息。

流 socket 使用的网络协议叫 TCP 即传输控制协议(Transmission Control Protocol)。数据报 socket 叫 UDP 即用户数据报协议(User Datagram Protocol)。它们的区别是什么呢? 何时选择何种 socket 是较好的呢? 在程序中如何使用数据报 socket 呢?

13.5.1 流与数据报的比较

socket 是如何工作的? 数据如何在 Internet 上传输的? 当用户向流 socket 写数据时, 内核要做什么? 这与向数据报 socket 写数据有何区别?

从一个流 socket 传输到另一个流 socket 的数据流, 看上去是连续的、无缝的, 实际上这是一种错觉。Internet 连接要把数据分割成独立的数据包。网络数据传输过程如图 13.4 所示。

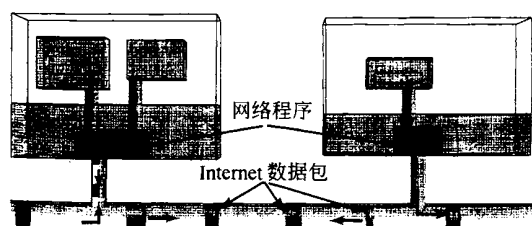


图 13.4 Internet 包传输数据

在现实世界中, 有很多将一大块数据分割成若干较小的数据块的例子。假设要通过快递传送 100 页的文档。当快递公司要求你使用只能装 20 页的信封时, 该怎么办? 可以把文档分割成 5 个包裹, 每个单独地打包和附上地址。然后把这 5 个独立的包裹放到邮箱中, 运输系统将负责把它们送到目的地。在接收端, 接受者打开这 5 个包裹, 并把它们按顺序重组成原来的文档。

Internet 就像上述运输系统一样, 它上面传输的数据必须符合大小的限制。大块的数据被分割成小块的数据来传输, 接收端必须以正确的顺序重组数据块。但通信过程可以被连接和中断, 如图 13.5 所示。

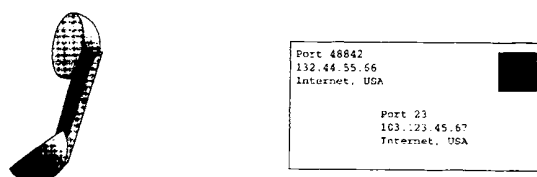


图 13.5 通信可以被连接和中断

流 socket 负责分割、排序、重组的所有工作。数据报 socket 则不会。下表给出了它们之间的区别。

TCP	UDP
流	数据报
分片/重组	否
排序	否
可靠的	可能未到达
连接的	多个发送者

在流 socket 中,内核将大的数据块分割成带编号的数据片。位于接收机器上的内核按顺序接收数据片,重组成发送者所发送的原始数据。在数据报 socket 中,内核并不给数据加编号标签,在目的地也不重组。

流 socket 对传送负责,数据报 socket 则不。流 socket 的接收端检查数据片的顺序来确保数据完整到达。接收端提示发送端丢失的数据片的编号,并等待丢失数据片的重传。数据报 socket 并不检查丢失数据报,也不要求重传。如果某个包丢失在 Internet 中,它只是不到达。TCP 比 UDP 做更多的工作。UDP 更快、更简单,给网络较少的负荷。

UDP 接收消息跟邮箱系统方式相同:发送者将你的地址写在信件上,邮件系统负责将信件送到你的邮箱中,然后你从邮箱中取出信。TCP socket 需要明确调用 accept、read 和 close 来读取远端进程的消息。

UDP 正好适合这里的应用。客户发送短消息来获得许可,服务器发送短消息给予或拒绝许可。客户和服务器的交互不需要建立连接,不需要分片和重组。可靠性甚至是不要的。如果请求或票据丢失,客户可以再次请求。在任何事件中,服务器和客户就像在一台机器上或者一个网络的同一部分,所以丢失数据报的风险较小。

UDP 对于 Web 服务器和 e-mail 服务是一个较差的选择。Web 服务器和 e-mail 信息可能是大的文件,这些字节流必须完全和按序到达目的地。UDP 对于允许丢失帧的声音和视频流是较好的选择。

13.5.2 数据报编程

数据报与邮件网络系统类似,包括 3 个主要部分:目的地址、返回地址和消息,如图 13.6 所示。

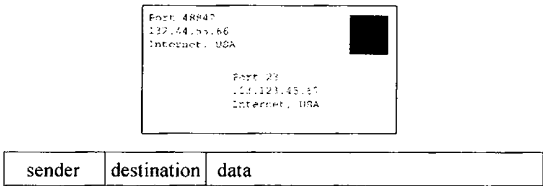
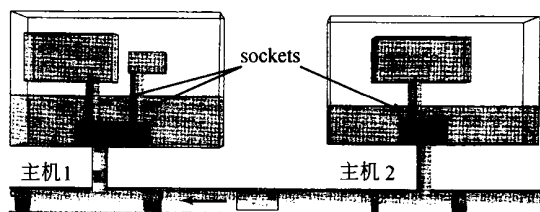


图 13.6 数据报的 3 部分

数据报 socket 可以被理解成一个内部有数据的盒子,发送者给出目的 socket 的地址。网络将数据从发送者发送到目的 socket。接收进程从该 socket 中读取数据。程序使用 sendto 发送数据和 recvfrom 来读取数据,如图 13.7 所示。



一台主机可能有多个接收 socket。每个 socket 被指派给一个特定的端口号。地址用主机上的端口来区分。

图 13.7 使用 sendto 和 recvfrom

1. 接收数据报

程序 dgreceiv.c 是一个简单的基于数据报的服务器。dgreceiv.c 使用命令行传过来的端口号建立 socket, 然后进入循环, 接收和打印从客户端发来的数据报:

```

/*****
 * dgreceiv.c - datagram receiver
 *
 *      usage: dgreceiv portnum
 *
 *      action: listens at the specified port and reports messages
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define oops(m,x) { perror(m);exit(x);}

int make_dgram_server_socket(int);
int get_internet_address(char *, int, int *, struct sockaddr_in *);
void say_who_called(struct sockaddr_in *);

int main(int ac, char * av[])
{
    int      port;                /* use this port */
    int      sock;                /* for this socket */
    char     buf[BUFSIZ];         /* to receive data here */
    size_t   msglen;              /* store its length here */
    struct   sockaddr_in saddr;    /* put sender's address here */
    socklen_t saddrlen;           /* and its length here */

    if ( ac == 1 || (port = atoi(av[1])) <= 0 ){
        fprintf(stderr, "usage: dgreceiv portnumber\n");
        exit(1);
    }

```

```

    }

    /* get a socket and assign it a port number */

    if( (sock = make_dgram_server_socket(port)) == -1 )
        oops("cannot make socket",2);

    /* receive messages on that socket */

    saddrlen = sizeof(saddr);
    while( (msglen = recvfrom(sock,buf,BUFSIZ,0,&saddr,&saddrlen))>0 )
    {
        buf[msglen] = '\0';
        printf("dgrecev: got a message: %s\n", buf);
        say_who_called(&saddr);
    }
    return 0;
}

void say_who_called(struct sockaddr_in * addrp)
{
    char    host[BUFSIZ];
    int     port;

    get_internet_address(host,BUFSIZ,&port,addrp);
    printf(" from: %s: %d\n", host, port);
}

```

辅助函数 `make_dgram_server_socket` 和 `get_internet_address` 将在后面给出的文件 `dgram.c` 中定义。在数据报 socket 中接收消息比从流 socket 接收简单。`recvfrom` 函数阻塞直到数据报到达。当数据报到达时,消息内容、返回地址和其长度将被复制到缓存中。

2. 发送数据报

程序 `dgsend.c` 发送数据报。`dgsend.c` 创建一个 socket,然后用它发送消息到以命令行参数传入的特定的主机和端口号。

```

/*****
 * dgsend.c - datagram sender
 *
 *          usage: dgsend hostname portnum "message"
 *          action: sends message to hostname:portnum
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define oops(m,x) { perror(m);exit(x);}

```

```

int make_dgram_client_socket();
int make_internet_address(char *,int, struct sockaddr_in *);

int main(int ac, char *av[])
{
    int     sock;                /* use this socket to send */
    char    *msg;                /* send this message */
    struct  sockaddr_in  saddr;   /* put sender's address here */

    if ( ac != 4 ){
        fprintf(stderr,"usage: dgsend host port 'message'\n");
        exit(1);
    }
    msg = av[3];

    /* get a datagram socket */

    if( (sock = make_dgram_client_socket()) == -1 )
        oops("cannot make socket",2);

    /* combine hostname and portnumber of destination into an address */

    make_internet_address(av[1], atoi(av[2]), &saddr)

    /* send a string through the socket to that address */

    if ( sendto(sock, msg, strlen(msg), 0, &saddr, sizeof(saddr)) == -1 )
        oops("sendto failed", 3);
    return 0;
}

```

sendto 函数将缓存中的内容发送到特定地址的 socket。

3. 辅助函数

创建 socket 和 socket 地址的细节被封装在 dgram.c 中:

```

/*****
 * dgram.c
 * support functions for datagram based programs
 */

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <string.h>

#define HOSTLEN 256

```

```

int make_internet_address();

int make_dgram_server_socket(int portnum)
{
    struct sockaddr_in saddr; /* build our address here */
    char    hostname[HOSTLEN]; /* address */
    int     sock_id;          /* the socket */

    sock_id = socket(PF_INET, SOCK_DGRAM, 0); /* get a socket */
    if ( sock_id == -1 ) return -1;

    /** build address and bind it to socket ** /

    gethostname(hostname, HOSTLEN); /* where am I ? */
    make_internet_address(hostname, portnum, &saddr);

    if( bind(sock_id, (struct sockaddr *)&saddr, sizeof(saddr)) != 0 )
        return -1;

    return sock_id;
}

int make_dgram_client_socket()
{
    return socket(PF_INET, SOCK_DGRAM, 0);
}

int make_internet_address(char * hostname, int port, struct sockaddr_in * addrp)
/*
 * constructor for an Internet socket address, uses hostname and port
 * (host,port) -> * addrp
 */
{
    struct hostent * hp;

    bzero((void *)addrp, sizeof(struct sockaddr_in));
    hp = gethostbyname(hostname);
    if ( hp == NULL ) return -1;
    bcopy((void *)hp->h_addr, (void *)&addrp->sin_addr, hp->h_length);
    addrp->sin_port = htons(port);
    addrp->sin_family = AF_INET;
    return 0;
}

int get_internet_address(char * host, int len, int * portp, struct sockaddr_in * addrp)
/*
 * extracts host and port from an internet socket address
 ** addrp -> (host,port)
 */
{
    strncpy(host, inet_ntoa(addrp->sin_addr), len );

```

```

    * portp = ntohs(addrp->sin_port);
    return 0;
}

```

创建一个数据报 socket 与创建一个流 socket 类似。其不同点在于,这里设置 socket 的类型为 SOCK_DGRAM,而且不要调用 listen 函数。

4. 编译和测试

```

$ cc dgrecv.c dgram.c -o dgrecv
$ ./dgrecv 4444 &
[1] 19383
$ cc dgsend.c dgram.c -o dgsend
$ ./dgsend host2 4444 "testing 123"
dgrecv:got a message: testing 123
      from:10.200.75.200:1041
$ ps
PID    TTY      TIME      CMD
14599  pts/3    00:00:00  bash
19383  pts/3    00:00:00  dgrecv
19393  pts/3    00:00:00  ps
$

```

编译服务器,并启动它,使得它监听在端口 4444。然后编译和运行客户,使客户发送字符串到端口 4444。服务器接收消息,打印消息,并且打印消息的返回地址。客户 socket 拥有主机地址和端口号,内核随机地给它分配了一个端口号 1041。ps 进程显示了服务器正在运行。

13.5.3 sendto 和 recvfrom 的小结

sendto		
目标	从 socket 发送消息	
头文件	#include <sys/types.h> #include <sys/socket.h>	
函数原型	nchars = sendto(int socket, const void * msg, size_t len, int flags, const struct sockaddr * dest, socklen_t dest_len);	
参数	socket	socket id
	msg	发送的字符类型的数组
	len	发送的字符数
	flags	比特的集合,设置发送属性,0 表示普通
	dest	指向远端 socket 地址的指针
	dest_len	地址长度
返回值	-1	遇到错误
	nchars	发送的字符数

sendto 从源 socket 发送数据到目的 socket。前 3 个参数与 write 的参数类似：要发送的 socket、保存要发送字符串的数组以及发送的字符数。与 write 类似，sendto 返回实际发送的字符数。flags 参数表明发送的各种属性；具体可参见 Unix 版本的帮助信息。最后两个参数给出发送目的地的 socket 地址。如果是 Internet 类型的地址，socket 地址包含目的主机的 IP 地址和端口号，而其他类型的地址则包含其他的成员。

recvfrom		
目标	从 socket 接收消息	
头文件	#include <sys/types.h> #include <sys/socket.h>	
函数原型	nchars = recvfrom(int socket, const void * msg, size_t len, int flags, const struct sockaddr * sender, socklen_t * sender_len);	
参数	socket	socket id
	msg	字符类型的数组
	len	接收的字符数
	flags	表示接收属性的比特的集合, 0 表示普通
	sender	指向远端 socket 的地址的指针
	sender_len	地址长度
返回值	-1	遇到错误
	nchars	接收的字符数

recvfrom 从 socket 读取数据报。前 3 个参数与 read 类似：所要读取的 socket、存放字符的数组以及要读取的字符数。与 read 类似，recvfrom 返回实际接收的字符数。flags 指出了接收时所用的各种属性；具体可参见 Unix 系统的帮助信息。通过最后两个参数，可以获得发送者的地址。发送 socket 的地址将被存放在由第一个参数指向的结构中，地址长度存放在由第二个参数指向的整型值中。地址的长度必须提供给 recvfrom 函数；如果实际的地址是不同的长度，它将更改该值。如果第一个参数指针为空值，发送者地址将不被记录。

13.5.4 数据报应答

程序 dgsend.c 和 dgrecv.c 显示了如何从客户发送数据给服务器。服务器如何给客户发送应答呢？在现实世界中，假设有人给你发送了一封晚宴的邀请函，你将如何给出答复呢？这很简单：你只要给邀请函上的返回地址回信就行了。

程序 dgrecv2.c 从客户处接收消息并且发送谢谢作为应答：

```

/*****
 * dgrecv2.c - datagram receiver
 *
 *      usage: dgrecv portnum
 *
 *      action: receives messages, prints them, sends reply
 */

#include <stdio.h>
#include <stdlib.h>

```



```

#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define oops(m,x) { perror(m);exit(x);}

int make_dgram_server_socket(int);
int get_internet_address(char*,int, int* struct sockaddr_in*);
void say_who_called(struct sockaddr_in *);
void reply_to_sender(int,char *,struct sockaddr_in *, socklen_t);

int main(int ac, char * av[])
{
    int      port;                /* use this port */
    int      sock;                /* for this socket */
    char      buf[BUFSIZ];        /* to receive data here */
    size_t    msglen;             /* store its length here */
    struct    sockaddr_in saddr;   /* put sender's address here */
    socklen_t saddrlen;           /* and its length here */

    if ( ac == 1 || (port = atoi(av[1])) <= 0 ){
        fprintf(stderr,"usage: dgre cv portnumber\n");
        exit(1);
    }

    /* get a socket and assign it a port number */

    if( (sock = make_dgram_server_socket(port)) == -1 )
        oops("cannot make socket",2);

    /* receive messaages on that socket */

    saddrlen = sizeof(saddr);
    while( (msglen = recvfrom(sock,buf,BUFSIZ,0, &saddr,&saddrlen))>0 ) {
        buf[msglen] = '\0';
        printf("dgre cv: got a message: %s\n", buf);
        say_who_called(&saddr);
        reply_to_sender(sock,buf,&saddr,saddrlen);
    }

    return 0;
}

void reply_to_sender(int sock,char * msg,struct sockaddr_in * addrp,socklen_t len)
{
    char reply[BUFSIZ + BUFSIZ];

    sprintf(reply, "Thanks for your %d char message\n", strlen(msg));

```

```

    sendto(sock, reply, strlen(reply), 0, addrp, len);
}
void say_who_called(struct sockaddr_in * addrp)
{
    char    host[BUFSIZ];
    int     port;

    get_internet_address(host, BUFSIZ, &port, addrp);
    printf(" from: %s: %d\n", host, port);
}

```

发送者程序当然也要改变以接收应答。这作为练习由读者来完成。

13.5.5 数据报小结

数据报是从一个 socket 发送到另一个的短消息。发送者使用 `sendto` 来指定消息、长度和目的地。接收者使用 `recvfrom` 接收消息。数据报和带有地址的 Internet 上传输的数据包的基本结构接近。因此,数据报给内核网络功能和网络流量增加的负荷较少。由于数据报可能在传输中丢失,也有可能不按顺序地到达,所以它通常用对简单和高效的要求比完整性和一致性更为重要的应用中。

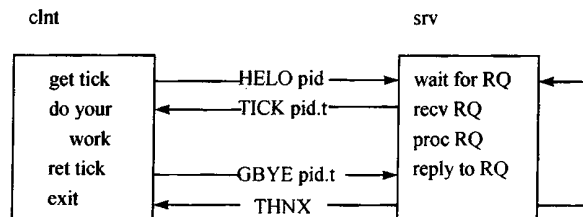
本章的许可证服务器是一种简单的消息传输,它用一个服务器来接收、处理短消息和发送请求,所以数据报是一个合适的选择。

13.6 许可证服务器版本 1.0

下面,回到许可证服务器项目上来。这里的服务器限制了程序同时运行的实例数目。当用户要运行受限制的程序时,该进程要向服务器请求运行许可。

如果并没有很多人正在用该程序,服务器发送票据给进程,给予许可证。如果达到了最大的程序实例数,服务器发送无可用票据的消息给客户,并且通知客户稍后再试或者购买支持更多用户版本的软件。被许可程序和服务器之间通过数据报来通信。

客户和服务器的运行流程及其交互过程如下。



客户和服务分别由两个文件组成:短的文件包含 `main` 函数,长的文件包含票据管理函数。下面将分析客户和服务程序。

13.6.1 客户端版本 1

```

/*****
 * lclnt1.c
 * License server client version 1
 * link with lclnt_funcs1.o dgram.o
 */

#include <stdio.h>

int main(int ac, char *av[])
{
    setup();
    if (get_ticket() != 0)
        exit(0);

    do_regular_work();

    release_ticket();
    shut_down();
}

/*****
 * do_regular_work the main work of the application goes here
 */
do_regular_work()
{
    printf("SuperSleep version 1.0 Running - Licensed Software\n");
    sleep(10); /* our patented sleep algorithm */
}

```

客户端的最上层代码遵照了上一节中所小结的原则。客户得到票据, 进行处理, 释放票据, 然后退出。该许可证例程是 Unix 中 sleep 程序的特殊版本, 若不满意标准 sleep 版本的工作, 也可以购买许可证来使用此版本的程序。当然还要运行许可证服务器, 否则该 sleep 程序将拒绝运行。其中辅助函数在 lclnt_funcs1.c 中:

```

/*****
 * lclnt_funcs1.c: functions for the client of the license server
 */

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

```

```

/*
 * Important variables used throughout
 */
static int pid = -1;           /* Our PID */
static int sd = -1;           /* Our communications socket */
static struct sockaddr serv_addr; /* Server address */
static socklen_t serv_alen;    /* length of address */
static char ticket_buf[128];   /* Buffer to hold our ticket */
static have_ticket = 0;        /* Set when we have a ticket */

#define MSGLEN 128             /* Size of our datagrams */
#define SERVER_PORTNUM 2020    /* Our server's port number */
#define HOSTLEN 512
#define oops(p) { perror(p); exit(1); }

char *do_transaction();

/*
 * setup: get pid, socket, and address of license server
 * IN no args
 * RET nothing, dies on error
 * notes: assumes server is on same host as client
 */
setup()
{
    char    hostname[BUFSIZ];

    pid = getpid();           /* for ticks and msgs */
    sd = make_dgram_client_socket(); /* to talk to server */
    if ( sd == -1 )
        oops("Cannot create socket");
    gethostname(hostname, HOSTLEN); /* server on same host */
    make_internet_address(hostname, SERVER_PORTNUM, &serv_addr);
    serv_alen = sizeof(serv_addr);
}

shut_down()
{
    close(sd);
}

/*****
 * get_ticket
 * get a ticket from the license server
 * Results: 0 for success, -1 for failure
 */
int get_ticket()

```

```

{
    char * response;
    char buf[MSGLEN];

    if(have_ticket)                                /* don't be greedy */
        return(0);

    sprintf(buf, "HELLO %d", pid);                  /* compose request */

    if ( (response = do_transaction(buf)) == NULL )
        return(-1);

    /* parse the response and see if we got a ticket.
     * on success, the message is: TICK ticket - string
     * on failure, the message is: FAIL failure - msg
     */
    if ( strcmp(response, "TICK", 4) == 0 ){
        strcpy(ticket_buf, response + 5);          /* grab ticket - id */
        have_ticket = 1;                           /* set this flag */
        narrate("got ticket", ticket_buf);
        return(0);
    }

    if ( strcmp(response, "FAIL", 4) == 0 )
        narrate("Could not get ticket", response);
    else
        narrate("Unknown message:", response);

    return(-1);
} /* get_ticket */

/*****
 * release_ticket
 * Give a ticket back to the server
 * Results: 0 for success, -1 for failure
 */
int release_ticket()
{
    char buf[MSGLEN];
    char * response;

    if(! have_ticket)                                /* don't have a ticket */
        return(0);                                    /* nothing to release */

    sprintf(buf, "GBYE %s", ticket_buf);            /* compose message */
    if ( (response = do_transaction(buf)) == NULL )
        return(-1);

    /* examine response

```

• 400 •

Unix/Linux 编程实践教程

```

    * success: THNX info - string
    * failure: FAIL error - string
    */
    if ( strcmp(response, "THNX", 4) == 0 ){
        narrate("released ticket OK", "");
        return 0;
    }
    if ( strcmp(response, "FAIL", 4) == 0)
        narrate("release failed", response + 5);
    else
        narrate("Unknown message:", response);
    return(-1);
} /* release_ticket */

/*****
 * do_transaction
 * Send a request to the server and get a response back
 * IN msg_p          message to send
 * Results: pointer to message string, or NULL for error
 *               NOTE: pointer returned is to static storage
 *               overwritten by each successive call.
 * note: for extra security, compare retaddr to serv_addr (why?)
 */
char * do_transaction(char * msg)
{
    static char buf[MSGLEN];
    struct sockaddr retaddr;
    socklen_t    addrlen = sizeof(retaddr);
    int ret;

    ret = sendto(sd, msg, strlen(msg), 0, &serv_addr, serv_alen);
    if ( ret == -1 ){
        syserr("sendto");
        return(NULL);
    }
    /* Get the response back */
    ret = recvfrom(sd, buf, MSGLEN, 0, &retaddr, &addrlen);
    if ( ret == -1 ){
        syserr("recvfrom");
        return(NULL);
    }

    /* Now return the message itself */
    return(buf);
}

```

```

} /* do_transaction */

/*****
 * narrate: print messages to stderr for debugging and demo purposes
 * IN msg1, msg2 : strings to print along with pid and title
 * RET nothing, dies on error
 */
narrate(char * msg1, char * msg2)
{
    fprintf(stderr, "CLIENT [ %d]: %s %s\n", pid, msg1, msg2);
}
syserr(char * msg1)
{
    char    buf[MSGLEN];
    sprintf(buf, "CLIENT [ %d]: %s", pid, msg1);
    perror(buf);
}

```

get_ticket 和 release_ticket 是程序中的主要函数。它们都遵循下面的规则: 产生短请求、发送消息给服务器、等待服务器的应答, 然后检查应答和根据应答采取行动。

get_ticket 通过发送命令 HELLO 以及紧跟其后的 PID 来请求票据。服务器通过发送 TICK ticket-id 接收请求。服务器发送 FAIL explanation 拒绝请求。

release-ticket 通过发送命令 GBYE ticket-id 返回票据。如果票据是合法的, 服务器将发送 THNX greeting 消息作为应答。如果票据不合法, 服务器发送 FAIL explanation 消息。

为何票据可能是不合法的? 本文的后面将讨论该问题。

13.6.2 服务器端版本 1

```

/*****
 * lserv1.c
 * License server server program version 1
 */

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <signal.h>
#include <sys/errno.h>
#define MSGLEN      128          /* Size of our datagrams */

int main(int ac, char * av[])
{
    struct sockaddr_in client_addr;
    socklen_t addrlen = sizeof(client_addr);

```

```

char    buf[MSGLEN];
int      ret;
int      sock;

sock = setup();

while(1) {
    addrlen = sizeof(client_addr);
    ret = recvfrom(sock, buf, MSGLEN, 0, &client_addr, &addrlen);
    if ( ret != -1 ) {
        buf[ret] = '\0';
        narrate("GOT:", buf, &client_addr);
        handle_request(buf, &client_addr, addrlen);
    }
    else if ( errno != EINTR )
        perror("recvfrom");
}
}

```

许可证服务器的主函数是一个循环,主要包括接收客户请求,处理请求和发送应答。其中处理请求的代码包含在 lserv_funcs1.c 文件中。

```

/*****
 * lsrv_funcs1.c
 * functions for the license server
 */

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <signal.h>
#include <sys/errno.h>

#define SERVER_PORTNUM 2020          /* Our server's port number */
#define MSGLEN        128           /* Size of our datagrams */
#define TICKET_AVAIL  0              /* Slot is available for use */
#define MAXUSERS      3             /* Only 3 users for us */
#define oops(x) { perror(x); exit(-1); }

/*****
 * Important variables
 */
int ticket_array[MAXUSERS];         /* Our ticket array */
int sd = -1;                        /* Our socket */
int num_tickets_out = 0;            /* Number of tickets outstanding */

```



```

char *do_hello();
char *do_goodbye();

/*****
 * setup() - initialize license server
 */
setup()
{
    sd = make_dgram_server_socket(SERVER_PORTNUM);
    if ( sd == -1 )
        oops("make socket");
    free_all_tickets();
    return sd;
}

free_all_tickets()
{
    int i;

    for(i = 0; i < MAXUSERS; i++)
        ticket_array[i] = TICKET_AVAIL;
}

/*****
 * shut_down() - close down license server
 */
shut_down()
{
    close(sd);
}

/*****
 * handle_request(request, clientaddr, addrlen)
 * branch on code in request
 */
handle_request(char *req, struct sockaddr_in *client, socklen_t addlen)
{
    char *response;
    int ret;

    /* act and compose a response */
    if ( strncmp(req, "HELO", 4) == 0 )
        response = do_hello(req);
    else if ( strncmp(req, "GBYE", 4) == 0 )
        response = do_goodbye(req);
    else
        response = "FAIL invalid request";
}

```

```

    /* send the response to the client */
    narrate("SAID:", response, client);
    ret = sendto(sd, response, strlen(response), 0, client, addlen);
    if (ret == -1)
        perror("SERVER sendto failed");
}

/*****
 * do_hello
 * Give out a ticket if any are available
 * IN msg_p          message received from client
 * Results: ptr to response
 * Note: return is in static buffer overwritten by each call
 * NOTE: return is in static buffer overwritten by each call
 */
char *do_hello(char *msg_p)
{
    int x;
    static char replybuf[MSGLEN];

    if(num_tickets_out >= MAXUSERS)
        return("FAIL no tickets available");
    /* else find a free ticket and give it to client */

    for(x = 0; x<MAXUSERS && ticket_array[x] != TICKET_AVAIL; x++) ;

    /* A sanity check - should never happen */
    if(x == MAXUSERS) {
        narrate("database corrupt", "", NULL);
        return("FAIL database corrupt");
    }

    /* Found a free ticket. Record "name" of user (pid) in array.
     * generate ticket of form: pid.slot
     */
    ticket_array[x] = atoi(msg_p + 5); /* get pid in msg */
    sprintf(replybuf, "TICK %d.%d", ticket_array[x], x);
    num_tickets_out++;
    return(replybuf);
} /* do_hello */

/*****
 * do_goodbye
 * Take back ticket client is returning
 * IN msg_p          message received from client
 * Results: ptr to response

```

```

    * Note: return is in static buffer over written by each call
    */
char * do_goodbye(char * msg_p)
{
    int pid, slot;          /* components of ticket */

    /* The user's giving us back a ticket. First we need to get
    * the ticket out of the message, which looks like:
    *
    * GBYE pid.slot
    */
    if((sscanf((msg_p + 5), "%d.%d", &pid, &slot) != 2) ||
        (ticket_array[slot] != pid)) {
        narrate("Bogus ticket", msg_p + 5, NULL);
        return("FAIL invalid ticket");
    }

    /* The ticket is valid. Release it. */
    ticket_array[slot] = TICKET_AVAIL;
    num_tickets_out--;

    /* Return response */
    return("THNX See ya!");
} /* do_goodbye */

/*****
 * narrate() - chatty news for debugging and logging purposes
 */
narrate(char * msg1, char * msg2, struct sockaddr_in * clientp)
{
    fprintf(stderr, "\t\tSERVER: %s %s ", msg1, msg2);
    if ( clientp )

        fprintf(stderr, "( %s: %d)", inet_ntoa(clientp->sin_addr),
            ntohs(clientp->sin_port) );
    puts('\n', stderr);
}

```

3 个重要的函数解释如下。

(1) handle_request

请求由 4 个字符的命令带一个参数构成。服务器先检查命令, 然后调用对应的函数。即使命令不合法, 服务器也必须发送应答, 否则客户会一直阻塞下去。

(2) do_hello

HELO 命令用来请求票据。服务器查找票据数组寻找空闲的项。如果某项的 PID 值为 0, 表示这是一个可用票据。服务器使用独立的变量 num_tickets_out 来节省时间。服务器

接收请求后,可以通过查找表来寻找空闲项,而该变量可以指明何时表已经满了,这样就不必做查找了。

(3) do_goodbye

GBYE 命令是返回票据的请求。票据是一个由 PID 和票据编号构成的字符串。服务器将票据的 PID 和票据编号与签出列表(sign-out list)中的值进行比较,如果数据一致,服务器从签出列表中清除该项的值并且致谢客户。如果不一致,则一定是什么地方发生了错误。

如果你是飞机场的检票员,如果某个客户给出存根的编号和名字在数据库中不存在,你就可能会问:“你是从哪里得到这张票的,你是谁?”后面将讨论伪造票据问题。下面来测试这个版本的程序。

13.6.3 测试版本 1

编译服务器端程序并在后台运行它。

```
$ cc lserv1.c lserv_funcs1.c dgram.c -o lserv1
$ ./lserv1&
[1] 25738
```

编译好客户端程序,然后同时运行 4 个实例:

```
$ cc lc1nt1.c lc1nt_funcs1.c dgram.c -o lc1nt1
$ ./lc1nt1 & ./lc1nt1 & ./lc1nt1 & ./lc1nt1 &

SERVER:GOT;HELO 25912(10.200.75.200;1053)
SERVER:SAID;TICK 25912.0(10.200.75.200;1053)
CLIENT[25912]:got ticket 25912.0
SuperSleep version 1.0 Running - Licensed Software
SERVER:GOT;HELO 25913(10.200.75.200;1054)
SERVER:SAID;TICK 25913.1(10.200.75.200;1054)
CLIENT[25913]:got ticket 25913.1
SuperSleep version 1.0 Running - Licensed Software
SERVER:GOT;HELO 25915(10.200.75.200;1055)
SERVER:SAID;TICK 25915.2(10.200.75.200;1055)
CLIENT[25915]:got ticket 25915.2
SuperSleep version 1.0 Running - Licensed Software
SERVER:GOT;HELO 25914(10.200.75.200;1059)
SERVER:SAID;FAIL no tickets available (10.200.75.200;1059)
CLIENT[25914]:Could not get ticket FAIL no tickets available (10.200.75.200;1059)
SERVER:GOT;GBYE 25912.0(10.200.75.200;1053)
SERVER: SAID;THNX See ya! (10.200.75.200;1053)
CLIENT[25912]:released ticket OK
SERVER:GOT;GBYE 25913.1(10.200.75.200;1054)
SERVER: SAID;THNX See ya! (10.200.75.200;1054)
CLIENT[25913]:released ticket OK
```

```

SERVER: GOT; GBYE 25915.2(10.200.75.200;1055)
SERVER: SAID; THNX See ya! (10.200.75.200;1055)
CLIENT[25915]: released ticket OK

```

实际运行的程序可能有非常不同的结果。尽管如此,从中可以看到服务器如何接收请求、给出票据以及客户端如何获取票据并开始工作的;可以看到进程 25914 没有得到票据。因为在该进程出现时,所有的票据都已经被占用了,而之前进程 25915 却得到了一张票据。如果运行该程序多次,可能会看到不同的结果。

13.6.4 进一步的工作

版本 1 的许可证服务器可以很好地工作了: 服务器处理请求并且维持持有票据的进程列表。客户可以从服务器得到票据。现在为止一切都正常。看起来这非常理想。不过现实世界并不总这样美好,软件及其使用者并不完全是你所期望的那样。可能出现什么错误呢? 该如何处理这些错误呢?

13.7 处理现实的问题

这里的许可证服务器能很好地工作,前提是所有的进程是正常工作的。有时,软件可能运行出错。如果 SuperSleep 程序被另外一个用户杀死了,或者程序发生段存取错误而被内核杀死了,该如何呢? 对于它所占用的票据该如何处理呢? 如果服务器崩溃了怎么办呢? 在服务器重启后又将发生什么呢?

现实世界中的程序必须能够处理异常崩溃。这里考虑两种情形: 客户端崩溃和服务器崩溃。

13.7.1 处理客户端崩溃

如果客户端崩溃,客户将不会归还票据,如图 13.8 所示。

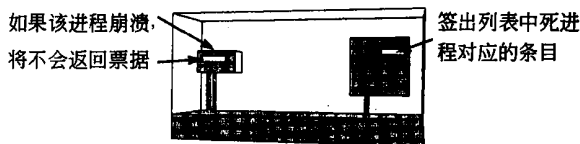


图 13.8 客户不归还票据

在出租车公司里,雇员可能辞职、被解聘、回家或死亡,但仍持有公司轿车的钥匙。这些对公司将造成什么影响呢? 签出列表指明票据仍被占用。其他进程就不能得到该票据。但是,如果有足够多的进程崩溃,签出列表就可能虽然满了,但此时却没有运行的客户程序。

钥匙管理员通过给持有钥匙的人打电话,就可以收回钥匙。他可以定期地浏览签出列表,然后给每个司机打电话,“你仍在使用车吗? ”。如果无人响应,管理员把他的名字从签出列表中划去。管理员检查的频率越高,签出列表的精确性就越高。

许可证服务器可以使用相同的技术,定期检查票据数组,确认其中的每个进程是否还活着? 如果某个进程已经不存在了,服务器可以把该进程从数组中去除,释放其占用的票据。检查程序运行的越频繁,数组的精确性越高。

1. 收回丢失的票据: 调度

服务器中如何增加收回票据的代码? 如何调用这些代码? 服务器必须实现两个独立的操作: 等待客户的请求,同时周期性地收回丢失的票据。而调度行为是简单的: 只要使用 alarm 和 signal 技术来周期地调用一个函数。在前面章节的移动文字例子中,使用了这种技术。修订的程序流程如图 13.9 所示。

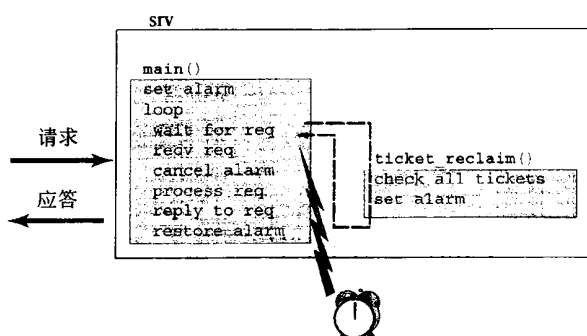


图 13.9 使用 alarm 来调度票据消除程序

在设计需同时处理两件事情的程序时,必定要考虑函数之间的冲突。如果服务器正在处理客户请求的同时,被 SIGALARM 信号触发调用回收丢失的票据的函数,会产生问题吗? 这两个处理函数共享变量或者数据结构吗? 显然是的,发放票据需要修改签出列表,而收回票据也需要修改签出列表。这种冲突可能会破坏数据的一致性吗? 该问题留作课后练习。考虑到安全性,在处理请求的时候关闭 alarm。

2. 收回丢失的票据: 编程

服务器希望能回收已经不存在进程的票据。那么如何判断进程是否还活着呢? 可以使用 popen 来运行 ps, 然后从 ps 的输出中查找 PID, 以确定持有票据的 PID 是否存在。另一种快速简洁的方法是使用 kill 系统调用的特殊功能。

可以通过给进程发送编号为 0 的信号以确定它是否存在。如果进程不存在,内核将不会发送信号,而是返回错误并设置 errno 为 ESRCH。在 ticket_reclaim 中使用了该特征,该函数在 lserv_funcs2.c 文件中:

```

#define RECLAIM_INTERVAL 60 /* reclaim every 60 seconds */
/*****
 * ticket_reclaim
 * go through all tickets and reclaim ones belonging to dead processes
 * Results: none
 */
void ticket_reclaim()
{

```

```

int i;
char tick[BUFSIZ];

for(i = 0; i < MAXUSERS; i++) {
    if((ticket_array[i] != TICKET_AVAIL) &&
        (kill(ticket_array[i], 0) == -1) && (errno == ESRCH)) {
        /* Process is gone - free up slot */
        sprintf(ticket, "%d. %d", ticket_array[i], i);
        narrate("freeing", ticket, NULL);
        ticket_array[i] = TICKET_AVAIL;
        num_tickets_out--;
    }
}

alarm(RECLAIM_INTERVAL);      /* reset alarm clock */
}

```

接下来,在 main 函数中增加调度回收票据的函数,并在正常操作中关闭 alarm。修改过的 main 在文件 lserv2.c 中:

```

int main(int ac, char *av[])
{
    struct sockaddr client_addr;
    socklen_t addrlen = sizeof(client_addr);
    char buf[MSGLEN];
    int ret, sock;
    void ticket_reclaim();          /* version 2 addition */
    unsigned time_left;

    sock = setup();
    signal(SIGALRM, ticket_reclaim); /* run ticket reclaimer */
    alarm(RECLAIM_INTERVAL);        /* after this delay */

    while(1) {
        addrlen = sizeof(client_addr);
        ret = recvfrom(sock, buf, MSGLEN, 0, &client_addr, &addrlen);
        if (ret != -1) {
            buf[ret] = '\0';
            narrate("GOT:", buf, &client_addr);
            time_left = alarm(0);
            handle_request(buf, &client_addr, addrlen);
            alarm(time_left);
        }
        else if (errno != EINTR)
            perror("recvfrom");
    }
}

```

通过上面的修改,许可证服务器就可以周期性地检查票据了。确实需要这样周期性地检查吗?为什么不只在票据列表满了并且有客户的请求被拒绝的时候检查呢?这样会更好吗?

13.7.2 处理服务器崩溃

服务器崩溃通常有两个严重的后果。首先,签出列表丢失,失去进程持有票据的记录。其次,新客户不可以再运行,因为分发许可证的程序已经不存在。最简单的解决方法是重新启动服务器,如图 13.10 所示。

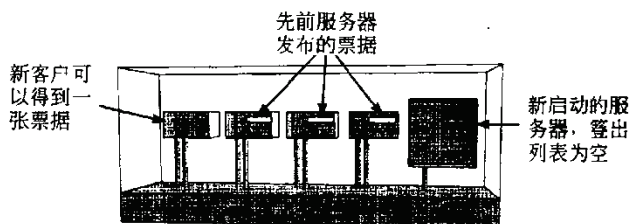


图 13.10 服务器在崩溃后重启

重启服务器使得新的客户可以运行,但会带来两个新的问题。

首先,重启服务器的票据数组是空的;服务器含有新的未被取走的票据列表。崩溃的服务器的票据数组可能已经满了,而重启服务器仍给其他客户发送许可。这样重复地关闭服务器,再重启服务器就像印钞机一样可以产生更多的票据。

其次,持有旧的服务器的票据的客户在归还票据时,将会被认为是伪造票据。

1. 票据验证

上述问题的一个解决方法是票据验证。票据验证表示每个客户周期性地向服务器发送票据的副本。客户发送数据包,对服务器说:“这是我的票据。是合法的吗?”,如图 13.11 所示。

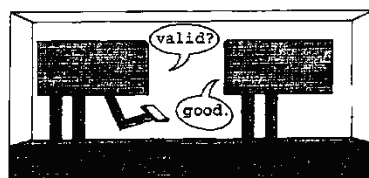


图 13.11 客户验证票据

票据含有数组编号和 PID。服务器检查签出列表。如果该项为空,服务器可以认为该票据是自己先前的实例赋予的。服务器将会把该票据加到列表中。逐步地,客户提供票据来验证,签出列表被重新填入。

服务器重建签出列表解决了表丢失问题,但可能导致其他问题。如果一个新的客户在表重建好之前,请求票据,服务器可能分发一个已经给予其他客户的票据给该客户。当持有旧的票据的客户对其票据进行验证时,服务器会拒绝它。

另一种方法是服务器拒绝表中没有的所有的票据。持有被拒票据的客户尝试再申请新的。这种方法是否更好?

2. 协议中增加验证

票据验证是协议中的一个新的事务:

CLIENT: VALD tickid

SERVER: GOOD or FAIL invalid ticket

这里必须改变客户和服务端以支持验证。

3. 客户端增加验证

客户端增加验证, 需要编写一个函数并在主函数中调用它, 其流程如图 13.12 所示。

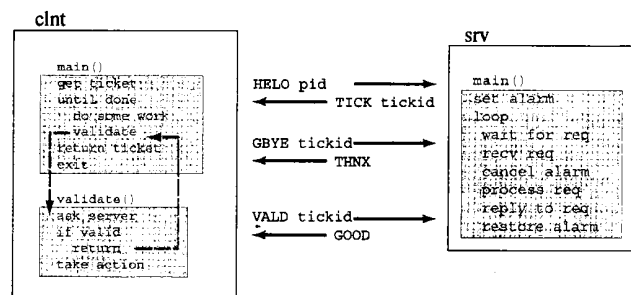


图 13.12 客户定期验证票据

客户可以根据系统的需要以一定的间隔定期验证票据, 可以设置一个计时器来定期验证。如果客户是一个电子制表程序, 验证可以在一定量的计算结束后进行。一个许可证服务器会响应验证请求。

SuperSleep 客户程序可以把 10 秒的睡眠时间分割成两个 5 秒, 在这期间进行验证。这作为课后练习。

4. 服务器端增加票据验证

服务器端增加验证需要做两处改动。改动后的程序位于新的文件 lserv_funcs2.c 中。首先, 增加一个函数来验证票据:

```

/*****
 * do_validate
 * Validate client's ticket
 * IN msg_p    message received from client
 * Results: ptr to response
 * NOTE: return is in static buffer overwritten by each call.
 */
static char *do_validate(char *msg)
{
    int pid, slot;          /* components of ticket */

    /* msg looks like VALD pid.slot - parse it and validate */
  
```

• 412 •

Unix/Linux 编程实践教程

```

    if (sscanf(msg + 5, "%d %d", &pid, &slot) == 2 && ticket_array[slot] == pid)
        return("GOOD Valid ticket");

    /* bad ticket */
    narrate("Bogus ticket", msg + 5, NULL);
    return("FAIL invalid ticket");
}

```

其次, `handle_request` 中增加更多的判断。

```

handle_request(char *req, struct sockaddr_in *client, socklen_t addlen)
{
    char *response;
    int ret;

    /* act and compose a response */
    if (strcmp(req, "HELLO", 4) == 0)
        response = do_hello(req);
    else if (strcmp(req, "GBYE", 4) == 0)
        response = do_goodbye(req);
    else if (strcmp(req, "VALID", 4) == 0)
        response = do_validate(req);
    else
        response = "FAIL invalid request";

    /* send the response to the client */
    narrate("SAID:", response, client);
    ret = sendto(sd, response, strlen(response), 0, client, addlen);
    if (ret == -1)
        perror("SERVER sendto failed");
}

```

13.7.3 测试版本 2

现在可以编译和测试新版本的客户和服务端了。测试包含了杀死客户和服务端以及重启客户和服务端。试观察输出中的进程 ID 和消息。为了便于测试,客户睡眠时间为两个 15 秒的间隔,服务器每 5 秒尝试回收票据。结果如下:

```

$ cc lserv2.c lserv_funcs2.c dgram.c -o lserv2
$ cc lcint2.c lcint_funcs2.c dgram.c -o lcint2
$ ./lserv2& #启动 1 个服务器
[1]30804
$ ./lcint2& ./lcint2& ./lcint2& #启动 3 个客户端
[2]30805
[3]30806

```

第13章 基于数据报(Datagram)的编程: 编写许可证服务器^①

• 413 •

```

[4]30807
$          SERVER: GOT: HELO 30805 (10.200.75.200;1085)
          SERVER: SAID: TICK 30805.0 (10.200.75.200;1085)
CLIENT [30805]:got ticket 30805.0
SuperSleep version 1.0 Running - Licensed Software
          SERVER: GOT: HELO 30806 (10.200.75.200;1086)
          SERVER: SAID: TICK 30806.1 (10.200.75.200;1086)
CLIENT [30806]:got ticket 30806.1
SuperSleep version 1.0 Running - Licensed Software
          SERVER: GOT: HELO 30807 (10.200.75.200;1087)
          SERVER: SAID: TICK 30807.2 (10.200.75.200;1087)
CLIENT [30807]:got ticket 30807.2
SuperSleep version 1.0 Running - Licensed Software
$ kill 30806 #kill 一个客户
[3]- Terminated      ./lclnt2
          SERVER: freeing 30806.1
          SERVER: GOT: VALD 30805.0 (10.200.75.200;1085)
          SERVER: SAID: GOOD Valid ticket (10.200.75.200;1085)
CLIENT [30805]:Validated ticket: GOOD Valid ticket
          SERVER: GOT : VALD 30807.2 (10.200.75.200;1087)
          SERVER: SAID: GOOD Valid ticket (10.200.75.200;1087)
CLIENT [30807]: Validated ticket: GOOD Valid ticket
$ kill 30804 #kill 服务器
[1]Terminated      ./lserv2
$ ./lserv2 # 启动新的服务器
[5]30808
$
          SERVER: GOT: GBYE 30805.0 (10.200.75.200;1085)
          SERVER: Bogus ticket 30805.0
          SERVER: SAID: FAIL invalid ticket (10.200.75.200;1085)
CLIENT [30805]:release failed invalid ticket
          SERVER: GOT: GBYE 30807.2 (10.200.75.200;1087)
          SERVER: Bogus ticket 30807.2
          SERVER: SAID: FAIL invalid ticket (10.200.75.200;1087)
CLIENT [30807]:release failed invalid ticket
$ ./lclnt2 # 启动一个新的客户
          SERVER: GOT: HELO 30809 (10.200.75.200;10857)
          SERVER: SAID: TICK 30809.0 (10.200.75.200;1087)
CLIENT [30809]:got ticket 30809.0
SuperSleep version 1.0 Running - Licensed Software
          SERVER: GOT : VALD 30809.0 (10.200.75.200;1087)
          SERVER: SAID: GOOD Valid ticket (10.200.75.200;1087)
CLIENT [30809]: Validated ticket: GOOD Valid ticket
          SERVER: GOT: GBYE 30809.0 (10.200.75.200;1087)

```

```
SERVER: SAID: THNX See ya! (10.200.75.200:1087)
CLIENT [30809];release ticket OK
[2] Done          ./lclnt2
[4] - Done        ./lclnt2
$ ps
PID   TTY     TIME     CMD
23509 pts/3   00:00:00 bash
30808 pts/3   00:00:00 lserv2
30810 pts/3   00:00:00 ps
$
```

看起来还不错。不妨试一下上述程序,观察其中的交互过程。

13.8 分布式许可证服务器

许可证服务器和被许可程序通过 socket 进行通信,socket 可以连接不同主机上的进程。理论上,与 Web 服务器和客户运行在不同主机上类似,这里的客户可以运行在一台机器上,而服务器运行在另一台机器上。当它们运行在不同的机器上时,会有问题吗?是的。

• 问题 1: 重复的进程 ID

进程 ID 在一台机器上是惟一的,但在不同主机上的进程可能拥有相同的进程 ID。图 13.13 说明的情况不含有任何错误且很常见。

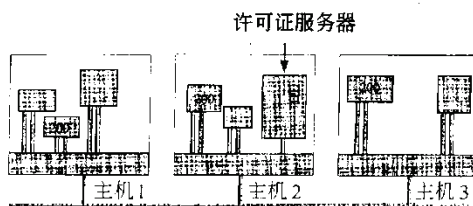


图 13.13 跨网络 PID 不惟一

存放票据的表中含有 PID 和票据编号。在图 13.13 的情况中,许可证服务将认为给同一个进程分配了 3 张票。每个进程只要一张票就可以运行,所以这是错误的。请求更多的票据,可以被认为是客户端的一个漏洞。

这里可以扩展票据表项的格式和内容,使其包含标识运行程序的主机从而解决重复 PID 问题。

• 问题 2: 回收票据

服务器通过调用 `kill(pid, 0)` 命令向客户回收票据。`kill(pid, 0)` 发送信号 0 给持有票据的进程。通过修订过的票据表,服务器现在可以知道客户运行在哪台主机上。

但是,服务器不能给其他机器上的进程发送信号,如图 13.14 所示。如果许可证服务器想给主机 3 上的进程发送信号,服务器必须产生主机 3 上的请求。

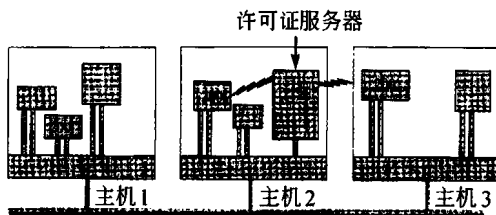


图 13.14 进程不能给其他主机发送信号

为什么不在每台机器上都运行一个服务器的实例？如图 13.15 所示，每个本地的服务器可以监控丢失的票据。

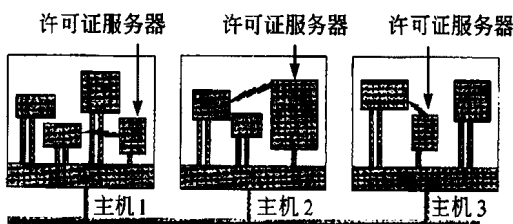


图 13.15 运行本地复制的 lserv

本地服务器解决了向主机发送信号的问题，但是又带来新的问题：哪个服务器发布票据？主服务器如何和本地服务器通信？客户把票据发给谁验证？

- 问题 3：主机崩溃

如果其中的一台机器停止运行，将会发生什么？主服务器如果还在运行的话，如何收回票据？客户端程序如何验证票据？如果运行主服务器的主机停止运行，谁来分发票据？

如何建立一个分布式许可证系统来同时支持多台机器？这里有 3 种方法。试考虑它们的设计细节以及优缺点，并考虑当客户、服务器、计算机或者网络崩溃时每个方案的后果。

- 方法 1：客户端服务器和中央服务器通信

每台机器都有一个本地服务器，就像本文编写的那样。每个客户跟本地的服务器通信。本地服务器把请求转发给中央服务器。中央服务器返回票据或者拒绝。本地服务器记录并把应答转发给客户。本地服务器也可以强制执行一些对本地客户的限制，例如该机器上可以运行的程序实例数，或者程序可以运行的时刻。

- 方法 2：每个客户都和中央服务器通信

客户直接给特定主机上的服务器发送请求。本地服务器运行在每个主机上，但这些服务器不和客户通信，它们在重新声明票据的时候作为中央服务器的代理。

- 方法 3：客户服务器和客户服务器通信

每台机器都有本地服务器，每个客户跟本地服务器通信。没有中央服务器。所有的本地服务器间互相通信。每次一个客户请求时，本地服务器询问其他所有的服务器目前已经用掉了多少张票。如果所用票据数小于所允许的总数，本地服务器分配一张票据给客户。

13.9 Unix 域 socket

本章的许可证服务器中的 socket 使用标准的主机 ID 和端口号地址系统。使用这些 Internet 地址,服务器可以接收本地的乃至更大网络上机器的客户请求。

在上述的两种分布式许可证服务器模型中,客户只需和同一台主机上的服务器通信。socket 只能用在仅限于主机内部的通信中吗?

13.9.1 文件名作为 socket 地址

有两种连接:流连接和数据报连接,也有两种 socket 地址:Internet 地址和本地地址。Internet 地址包含主机 ID 和端口号。本地地址通常叫做 Unix 域地址,它是一个文件名(例如/dev/log、/dev/printer 或/tmp/lserversock),没有主机和端口号。

两个 socket 名/dev/log 和/dev/printer 被用在很多 Unix 系统中。/dev/log 被 syslogd 服务器所使用。想要记录日志的程序只要给地址为/dev/log 的 socket 发送数据报就行了。地址/dev/printer 被一些打印系统使用。

13.9.2 使用 Unix 域 socket 编程

为了学习 Unix 域 socket 的客户/服务器编程,这里编写了一个日志系统。文件 wtmp 就是日志系统的一个实例。文件 wtmp 记录系统的所有登录、退出以及连接。日志系统被保证安全和系统维护的程序所使用,用来记录一些可疑行为。日志服务器是一个抄写员;客户发送信息给服务器,服务器将这些信息保存到只有自己可以修改的文件中。日志服务器可以用任何格式保存该文件,客户并不知道这些细节。

这里的日志服务器使用 Unix 域 socket 地址。只有同一台主机上的客户才能发消息给它。下面是客户和服务器的代码。服务器先创建 socket,然后绑定地址:

```
/* *****  
 * logfiled.c - a simple logfile server using Unix Domain Datagram Sockets  
 * usage: logfiled >>logfilename  
 */  
  
#include <stdio.h>  
#include <sys/types.h>  
#include <sys/socket.h>  
#include <sys/un.h>  
#include <time.h>  
  
#define MSGLEN 512  
#define oops(m,x) { perror(m); exit(x); }  
#define SOCKNAME "/tmp/logfilesock"  
  
int main(int ac, char *av[])  
{
```

```

int          sock;                                /* read messages here */
struct       sockaddr_un addr;                    /* this is its address */
socklen_t    addrlen;
char         msg[MSGLEN];
int          l;
char         sockname[] = SOCKNAME;
time_t       now;
int          msgnum = 0;
char         * timestr;

/* build an address */
addr.sun_family = AF_UNIX;                        /* note AF_UNIX */
strcpy(addr.sun_path, sockname);                  /* filename is address */
addrlen = strlen(sockname) + sizeof(addr.sun_family);

sock = socket(PF_UNIX, SOCK_DGRAM, 0);             /* note PF_UNIX */
if ( sock == -1 )
    oops("socket", 2);

/* bind the address */
if ( bind(sock, (struct sockaddr *) &addr, addrlen) == -1 )
    oops("bind", 3);

/* read and write */
while(1)
{
    l = read(sock, msg, MSGLEN);                  /* read works for DGRAM */
    msg[l] = '\0';                                /* make it a string */
    time(&now);
    timestr = ctime(&now);
    timestr[strlen(timestr)-1] = '\0';             /* chop newline */

    printf("[% 5d] %s %s\n", msgnum++, timestr, msg);
    fflush(stdout);
}
}

```

这里仍使用 socket 和 bind 来创建服务器 socket。socket 的类型是 SOCK_DGRAM, 地址族是 PF_UNIX^①。socket 地址是文件名。使用 read 而不是 recvfrom, 因为不需要应答。下面是客户端程序:

```

/*****
 * logfile.c - logfile client - send messages to the logfile server
 * usage: logfilec "a message here"
 */

```

^① PF_LOCAL 也许能替代 PF_UNIX。

• 418 •

Unix/Linux 编程实践教程

```

*/

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET "/tmp/logfilesock"
#define oops(m,x) { perror(m); exit(x); }

main(int ac, char *av[])
{
    int sock;
    struct sockaddr_un addr;
    socklen_t addrlen;
    char sockname[] = SOCKET;
    char *msg = av[1];

    if (ac != 2){
        fprintf(stderr, "usage: logfilec 'message'\n");
        exit(1);
    }

    sock = socket(PF_UNIX, SOCK_DGRAM, 0);
    if (sock == -1)
        oops("socket", 2);

    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, sockname);
    addrlen = strlen(sockname) + sizeof(addr.sun_family);

    if (sendto(sock, msg, strlen(msg), 0, &addr, addrlen) == -1)
        oops("sendto", 3);
}

```

这里使用 socket 函数来创建 socket, 并且使用 sendto 发送消息。服务器接收消息, 然后打印消息。消息前面是消息编号和时间。

下面是测试的情形:

```

$ cc logfiled.c -o logfiled
$ ./logfiled >>> visitorlog&
1500
$ cc logfilec.c -o logfilec
$ ./logfilec 'Nice system. Swell software!'
$ ./logfilec "Testing this log thing."
$ ./logfilec "Can you read this?"
$ cat visitorlog
[ 0] Mon Aug 20 18:25:34 2001 Nice system. Swell software!

```



```
[ 1] Mon Aug 20 18:25:44 2001 Testing this log thing.
[ 2] Mon Aug 20 18:25:48 2001 Can you read this?
```

上述两个程序展示了如何使用 Unix 域 socket 以及日志服务器的基本思想。程序的另一个特征是实现了自动追加功能,但没有使用 O_APPEND。服务器接收消息,然后把消息追加到文件末尾。即使有多个客户同时发送消息,底层的 socket 机制会负责消息的序列化。

13.10 小结: socket 和服务

socket 对于进程间的数据通信是强大的、万能的工具。这里学习了两种 socket 和两种 socket 地址。

socket	域	
	PF_INET	PF_UNIX
SOCK_STREAM	连接的,跨机器	连接的,本地
SOCK_DGRAM	数据报,跨机器	数据报,本地

在前面的几章中,已经学习了使用这 4 种组合中的 3 种 socket 的项目。当考虑 Unix 的网络编程和准备设计自己的项目时,可以考虑使用这张表格中的技术。根据发送消息的类型以及消息发送的距离来选择最佳的技术。

小 结

1. 主要内容

- 数据报是从一个 socket 发送到另一个 socket 的短消息。数据报 socket 是不连接的,每个消息包含有目的地址。数据报(UDP)socket 更加简单、快速,给系统增加的负荷更小。
- 许可证服务器是用来对被许可程序实施许可证验证规则的。许可证服务器发布许可,以短消息的形式发送给客户。
- 许可证服务器必须记住哪个进程使用了哪张票据,必须维持一个内部的数据库。因此,许可证服务器不同于本书的 Web 服务器。
- 记录系统状态的服务器必须设计成可以处理服务器和客户端的崩溃事件。
- 有些许可证服务器为一个网络上的多个机器提供服务。有几种设计方法,各有优缺点。
- socket 可以有两种类型的地址:网络或本地。本地的 socket 地址叫做 Unix 域 socket 或名字 socket。这种 socket 使用文件名作为地址,只能在一台机器上交互数据。

2. 下一步的工作

本章学习了两种服务器处理多个请求的方法。许可证服务器接收数据报请求,并且一次一条地返回消息。Web 服务器接收数据流消息,并且使用 fork 同时应答所有请求。服务

器有另外的选择：一个单一进程可以使用线程的技术同时运行几个函数。下一章将学习有关线程的思想和技术。

3. 习题

- 13.1 在使用流 socket 的例子中,服务器给客户应答时,并没有使用客户端的地址。服务器是如何知道给哪个地址的客户发消息的?
- 13.2 进程 25915 是如何战胜进程 25914 得到票据的? 考虑每个客户进程的创建和服务器端请求到达的操作序列。在多任务系统中,进程依次运行。进程在哪儿可以被中断从而得到测试的结果?
- 13.3 如何使用一个许可证服务器处理 2 个或更多的程序的请求? 一种方法是修改协议使得每个请求包含所要运行的程序名。描述可以支持多程序协议的数据结构和程序逻辑。
- 13.4 当服务器正在处理客户请求时,如果函数 ticket_reclaim 被调用,是否会对票据列表存在潜在的破坏? 考虑函数中每个更改数组和计数器的地方。在哪一点上数组的状态和计数器的值不一致? 处理函数是如何修改数组和计数器的? 这些值的一个未预测的改变对常规处理函数有何影响?
- 13.5 当进程被创建时,进程 ID 被分配给进程。考虑下面的时间序列: 一个进程 ID 为 777 的客户得到票据后崩溃了。不久,一个不同的用户运行程序创建了一个新的进程,进程 ID 恰好也是 777。当 ticket_reclaim 程序运行时,它发现了进程 777。即使现在编号为 777 的进程是一个不相关的程序,并且不持有票据,分配给原来进程 777 的票据也不能被回收。当这种情形出现时,该如何处理? 如何避免这类事件的发生。
- 13.6 一种防止票据数组丢失的方法是服务器将数据写到磁盘文件中。如果要适应这种备份机制,该如何改变服务器? 假设客户会故意杀死服务器以得到更多的票据,那么这里的文件备份机制在此种情形下该如何工作?
- 13.7 持有早期版本票据的客户在等待了较长的时间后来验证票据,可能发现已经没有了更多的可用票据了。为这种情形设计一种应答,使得客户不允许继续运行,因为这将破坏最大同时运行进程的数目,但要求客户不能突然退出。
- 13.8 参照本章中列出的问题,比较三种分布式许可证服务器模型。
- 13.9 使用 socket 时,可以用 write 和 sendto 来发送数据。阅读 send 和 sendmsg 的帮助手册,后面两个函数与前面的有什么区别?
- 13.10 轿车管理系统与数据报不只是比喻。设想每部轿车都装有 GPS 设备。一个计算机可以通过 modem 连接到 Internet,使得可以定位轿车的位置。设想轿车的启动不是受钥匙控制,而是通过一个磁卡的登录来控制。设计一个允许雇员登

录到系统来使用轿车,并且管理员可以跟踪司机行驶路线以及定位轿车位置的系统。

4. 编程练习

- 13.11 更改程序 `dgrece.c`, 要求不仅打印出发送者的地址和消息接收的时间, 还要打印消息编号。消息标号从 0 开始。希望得到的输出如下:

```
dgrece: got a message : testing 123
from: 10.200.75.200;1041
at :Sun Aug 19 10:22:27 EDT 2001
msg# :23
```

- 13.12 为 `dgsend.c` 增加客户程序 `dgrece2.c`。
- 13.13 许可证服务器在一张表中存有客户拥有票据的信息。如果想要服务器打印这些信息, 该如何操作? 看到表的信息有助于排错或测试服务器。一种标准的和服务器通信技术是使用信号。
更改程序 `lserv1` 使得它在接收信号 `SIGHUP` 时, 打印出表的内容。可以通过命令 `kill -HUP serverpid` 来测试该特征。
- 13.14 更改许可证服务器使得它只在一个客户的请求被拒绝时, 才调用 `ticket_reclaim` 程序。该方法的优缺点各是什么?
- 13.15 更改 `lcint2.c` 程序, 使它睡眠 5 秒钟后验证票据。如果票据合法, 客户再睡眠 5 秒钟, 然后退出并归还票据。如果票据不合法, 客户尝试请求另一个票据。如果成功, 继续正常操作。如果失败, 告诉用户许可证服务器出错然后再退出。
- 13.16 更改前面章节中编写的 `shell` 程序和 `bounce` 程序, 使用许可证服务器。在哪增加票据验证过程? 当服务器崩溃时, 票据不可用了, 该和用户说什么?
- 13.17 更改客户服务器代码使得票据包含主机 IP 地址。如何改变票据列表? 确信对验证函数也做了改变。
- 13.18 实现三种分布式许可证控制模型的一种。
- 13.19 日志系统的一个问题是其中的消息都是匿名的, 更改该系统使得消息包含发送消息用户的用户名。
- 13.20 在日志服务器中使用 `read`。编写两个新版本的服务器, 一个使用 `recvfrom`, 另一个使用 `recv`。这些获取数据的方法有何不同? 更详细的信息请阅读帮助手册。
- 13.21 如果许可证服务器和客户需要使用 Unix 域 socket, 需要做何改变? 解释客户为什么要使用 `bind`。

13.22 在第 1 章中,讨论了一个 Internet 桥牌游戏。在任何的分布式扑克游戏中,软件必须模拟一个扑克牌栈,以确保两个客户不会持有相同的扑克。

编写一对程序 `cardd` 和 `cardc`,使用数据报来处理一桌的扑克牌。启动时服务先洗牌,接下来客户从命令行启动,就可以从服务器获取扑克牌。例程运行如下:

```
$ cardc get 5
4D AH 2D TD KC
```

结果显示客户得到了 5 张牌,一张方块 4,一张红桃 A,一张方块 2,一张方块 10,一张王。确保程序不会将相同的扑克牌发给两个用户机,并且协议要有途径表明 dealer 何时发完牌。考虑一下还有其他什么有用的事务可以作为协议的补充?

5. 项目

基于本章的内容,可以学习编写下面的 Unix 程序:

`talk`、`rwho`、流媒体服务器